

JHU WebSec SQL Injection Module Assignment

v.25.1

Objectives for this Assignment

Show proficiency in implementing a web application server with SQL server functionality by deploying the LAMP stack.

Expand on the security challenges confronted by web application servers, particularly focusing on SQL Injection. Conduct a SQL Injection attack against your database and explain your method concisely.

Apply basic security principles to web server application technologies by writing a paragraph explaining why you were unable to complete the SQL injection attack successfully.

Introduction

In this assignment, you will set up a LAMP stack, which involves installing Apache (web server), MySQL (database), and PHP (application language) on an Ubuntu Linux server.

This is a widely used stack for web applications. Once your environment is set up, you will load data into your MySQL database and develop a small PHP application to attempt an SQL injection attack. CAUTION: These exercises are intended for educational purposes only and should be conducted within your local computer sandbox environment. (Note: "MySQLi" refers to the "improved" version of MySQL, while "SQLi" is commonly used as shorthand for "SQL Injection.")

Before You Begin

If you are using the Student Ubuntu VM, it is recommended you revert to a prior snapshot or use a fresh VM.

Update your VM

```
$ sudo apt update -y
```

Add an entry to **/etc/hosts** for local domain resolution for this module.

```
$ echo -e "\n\n127.0.0.1 websec.jhu" | sudo tee -a /etc/hosts > /dev/null
```

(Do you want to know what modifying `/etc/hosts` did for you? Check this out: [What is the Purpose of /etc/hosts File in Linux](#))

Verify the new hostname

```
$ ping -c 4 websec.jhu
```

Review the contents of **/etc/hosts**

```
$ more /etc/hosts
```

1. LAMP Stack and Database Setup

Install Apache server

From the command line, you will install Apache.

```
$ sudo apt install apache2
```

Right out of the box, Apache will be running. From inside your VM, open your Browser and navigate to **http://websec.jhu**

You will see the default webpage served by Apache.



Figure 1 OpenSSH running on Student VM

Install MySQL

Now we install MySQL, as the point of this module is to perform SQL injections.

```
$ sudo apt install mysql-server
```

Secure the database

```
$ sudo mysql

mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'password';
mysql> flush privileges;
mysql> exit

$ sudo mysql_secure_installation
```

Use these settings for securing your MySQL server

- Setup VALIDATE PASSWORD COMPONENT: **N**
- Change the password for root: **N**
- Remove anonymous users: **N**
- Disallow root login remotely: **N**
- Remove test database and access to it: **N**
- Reload privilege tables now: **Y**

Install PHP

Apache is installed to serve your content, while MySQL is configured to store and manage your data. PHP is the component responsible for processing code to deliver dynamic content to users. In addition to the PHP package, you will need to install **php-mysql**, a PHP module that enables communication between PHP and MySQL databases. The **libapache2-mod-php** module is also necessary to allow Apache to handle PHP files. Core PHP packages will be automatically installed as dependencies.

```
$ sudo apt install php libapache2-mod-php php-mysql php-cli
```

Verify your PHP Version

```
$ php -v
```

Create a Test PHP for Apache

```
$ echo -e "<?php \n phpinfo();" | sudo tee -a /var/www/html/info.php > /dev/null
```

Review the contents of **info.php**

```
$ more /var/www/html/info.php
```

Load the test SQL database

You can find information on the **employees** database here: [MySQL :: Employees Sample Database](#)

```
$ git clone https://github.com/datacharmer/test_db.git
$ cd test_db
$ sudo mysql -t -p < employees.sql
```

NOTE: Creating a non-root account (i.e., one without root privileges) may help you conduct tests in a manner that reflects how a normal user operates. This approach ensures that the results are realistic from the user's perspective.

Next we provide access to the **employee** database for a user who will run queries. You can use the **student** account or another account.

```
$ sudo mysql -p
mysql> CREATE USER 'student'@'localhost' IDENTIFIED BY 'student';
mysql> GRANT ALL ON employees.* TO 'student'@'localhost';
mysql> flush privileges;
mysql> exit;
```

Access MySQL and verify you can see the **employees** database

```
$ mysql -u student -p
mysql> show databases;
```

Part 1 Deliverables in a Word doc:

Screen snips of:

1. Status of Apache2

```
$ systemctl status apache2
```

2. Your Apache2 default page: <http://websec.jhu/>
3. Status of MySQL

```
$ systemctl status mysql
```

4. PHP info page: <http://websec.jhu/info.php>

2. SQL Injection

To carry out an SQL injection attack on your database, you will establish a web interface for the data. We have provided a basic web interface for you here: [jhuwebsec/JHU-WebSec-Mod4-interface.html](https://github.com/jhuwebsec/JHU-WebSec-Mod4-interface.html) at main · drcjhu/jhuwebsec, but you also have the option to create your own if you prefer.

All of the data is very interesting, but you will focus on the **salaries** table.

- A. Refer to the site, [PHP MySQL Select Data \(w3schools.com\)](https://www.w3schools.com/php/php_mysql_select_data.asp), to create a PHP script (within an HTML page) that retrieves all (emp no, salary) pairs from your database and displays them on the web page.
- B. Create an HTML page with a small form containing a textbox. After clicking the Submit button, the user can input an employee number.
 - a. Use the form we provide. Be sure to rename **module4.html** to **module4.php**.
 - b. OR use this site: [PHP Form Handling \(w3schools.com\)](https://www.w3schools.com/php/php_form_handling.asp) to create your own.
- C. Modify your query from the first step slightly to use a 'WHERE' clause. This will ensure that your query returns only the salary for the corresponding employee number.
- D. Once the previous step has been completed, try 3 different inputs to your employee number input field that could potentially trigger a SQL injection attack into the Employees database.

Optional references

- [SQL Injection | pentestmonkey](#)
- [SQL Injection Cheat Sheet | Invicti](#)
- [SQL injection cheat sheet | Web Security Academy \(portswigger.net\)](#)
- [What is SQL Injection? Tutorial & Examples | Web Security Academy \(portswigger.net\)](#)

Part 2 Deliverables in a Word doc:

- A. Provide the annotated screen snips of your 3 attempts to conduct SQL injections.
- B. For each of the SQLi queries you selected, briefly explain:
 - i. Why did you choose this query?
 - ii. What were your expected results?
 - iii. What was the actual result?

3. Security Analysis

You may or may not have been able to pull off your SQL injection attack. Note that you are likely using the MySQLi library and PHP 8.

Part 3 Deliverable add to the Word doc:

Write up a paragraph explaining why you were able to or couldn't successfully perform a SQL injection attack. Reflect on your experience with this assignment, and if necessary, conduct additional research on MySQLi and PHP 8 to enhance your understanding.

Grading Rubric:

Objective	Value
1. Demonstrate proficiency implementing a Web application server with SQL server functionality – Implemented LAMP stack, provided screen snips in the Word Doc of: 1. “\$ sudo service apache2 status” results 2. Apache server page 3. “\$ sudo service mysql status” results 4. PHP Info page.	20
2. Elaborate on specific security challenges faced by web application servers specifically SQL Injection. – submitted in a Word doc as follows:	
a. Three attempted a SQL Injection attacks against the database, provided annotated screen snips.	20
b. why you chose it c. your expected results d. what was the actual results.	25
3. Relate basic security fundamentals to Web server application technologies – Provided a paragraph explaining why were/weren’t able to successfully perform a SQL injection attack. - in the Word doc .	25
Does the submission look professional, easy to read, clean presentation, with spelling and grammar at master’s level?	10
TOTAL	100